Föreläsning 6

Rekursion och backtracking

Föreläsning 6

- Rekursiv lista
- Hissen i lustiga huset
- Bredden först med hjälp av kö
- Tornet i Hanoi
- Backtracking

Rekursiv toString i vår länkade lista

De flesta av våra metoder i vår länkade lista kan skrivas rekursivt. Dock blir de oftast långsammare. Prova gärna att skriva om några stycken som träning. Som exempel tar vi här upp toString (fler exempel finns i boken).

```
public String toString(){
    if(head!=null) return "["+toString(head)+"]";
    return "[]";
}
```

```
public String toString(Node<E> p){
    if(p.next!=null) return p.data.toString()+", "+toString(p.next);
    return p.data.toString();
}
```

Rekursivt definierad länkad lista

Det är inte bara metoderna i våra datastrukturer som kan skrivas rekursiva utan datastrukturer kan definieras rekursivt.

En länkad lista kan definieras rekursivt enligt:

En länkad lista är antingen tom eller består av en nod med en referens till ett data och en referens till en länkad lista.

Hissen i lustiga huset

Huset har *n* antal våningar (bottenvåningen som räknas som en av dessa kallas våning 1).

Hissen har två knappar:

den ena skickar upp hissen *upp* antal våningar om möjligt

den andra skickar ned hissen *ned* antal våningar om möjligt.

Om man befinner sig på bottenvåning hur många resor måste man då göra för att komma till våning *destination*?

Skriv en rekursiv funktion som tar reda på minsta antalet resor för valfritt värde på variablerna *n, upp, ned* och *destination*.

Exempel:

n=78, upp=15, ned=8, destination=35 ger minsta antalet resor till 13

Skal

```
static int nrOfTrips(int levels, int up, int down, int destination) {
    return nrOfTrips(levels, up, down, destination, 1);
}
```

```
private static int nrOfTrips(int levels, int up, int down, int destination, int position) {

}
```

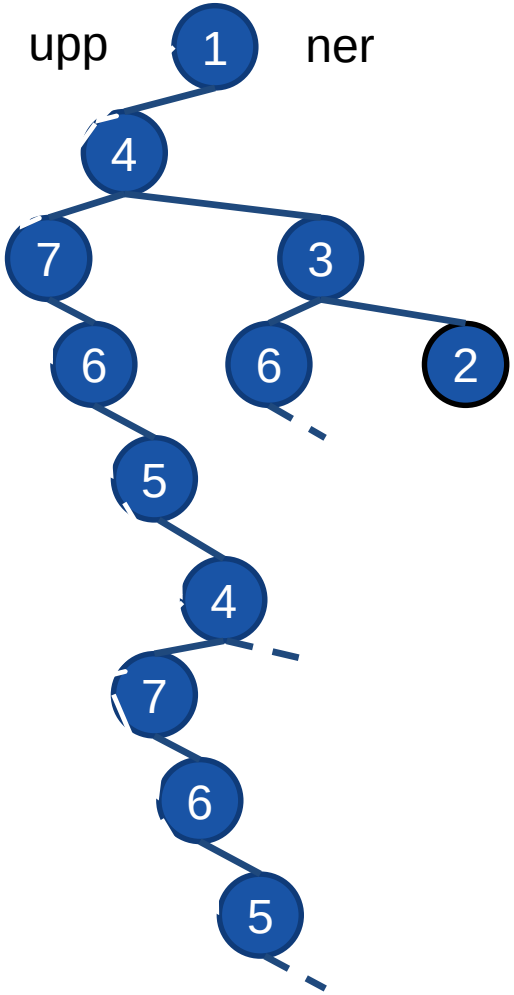
Första naivt förslag

```
private static int nrOfTrips(int levels, int up, int down, int destination, int position) {
    if (position == destination) return 0;

    int nrOfUp = Integer.MAX_VALUE, nrOfDown = Integer.MAX_VALUE;
    if (position + up <= levels)
            nrOfUp = 1 + nrOftrips(levels, up, down, destination, position + up);
    if(position - down >= 1)
            nrOfDown = 1 + nrOftrips(levels, up, down, destination, position - down);

    return Math.min(nrOfUp, nrOfDown);
}
```

Djupet först



Låt oss titta på vad som händer i ett enkelt exempel:

n = 7, upp = 3, ned = 1, destination = 2

Vårt program kommer åka:

1, 4, 7, 6, 5, 4, 7, 6, 5, 4, 7, osv

Här måste vi sätta ett maxdjup för att någonsin komma fram.

Nu med maxdjup

```
private static int nrOfTrips(int levels, int up, int down, int destination, int position, int trips) {
    if (position == destination) return 0;

    if (trips >= 30) return Integer.MAX_VALUE / 2;

    int nrOfUp = Integer.MAX_VALUE / 2, nrOfDown = Integer.MAX_VALUE / 2;
    if (position + up <= levels)
            nrOfUp = 1 + nrOftrips(levels, up, down, destination, position + up, trips + 1);
    if(position - down >= 1)
            nrOfDown = 1 + nrOftrips(levels, up, down, destination, position - down, trips + 1);

    return Math.min(nrOfUp, nrOfDown);
}
```

Varför skickar vi MAX/2 när vi ger upp?

Wrapper:

```
static int nrOfTrips(int levels, int up, int down, int destination) {
    return nrOfTrips(levels, up, down, destination, 1, 0);
}
```

Lösning som returnerar hela vägen

```
static int nrOfTrips2(int levels, int up, int down, int destination, int position, int trips) {
    if (position == destination) return trips;
    if (trips > 30) return Integer.MAX_VALUE;

    int nrOfUp = Integer.MAX_VALUE, nrOfDown = Integer.MAX_VALUE;
    if (position + up <= levels) nrOfUp = nrOfTrips2(levels, up, down, destination, position + up, trips + 1);
    if (position - down >= 1) nrOfDown = nrOfTrips2(levels, up, down, destination, position - down, trips + 1);

    return Math.min(nrOfUp, nrOfDown);
}
```

Observera att denna inte behöver returnera MAX/2. Den är dock sämre. Varje anrop svarar inte på ett delproblem så metodens namn är mindre korrekt och den går inte att effektivisera med dynamisk programmering.

```
static int nrOfTrips2(int levels, int up, int down, int destination) {
    return nrOfTrips2(levels, up, down, destination, 1, 0);
}
```

Avbryta tidigare?

Dock har vår senaste variant en fördel. När vi väl har hittat en lösning, säg med 10 resor så är det ju onödigt att leta efter nya lösningar som är längre än 9. Denna förbättring är något lättare att göra för den senaste lösningen:

```
static int nrOfTrips2b(int levels, int up, int down, int destination, int position, int trips, int bestSofar) {
    if (position == destination) return trips;
    if (trips >= bestSofar) return Integer.MAX_VALUE;
    int nrOfUp = Integer.MAX_VALUE, nrOfDown = Integer.MAX_VALUE;
    if (position + up <= levels)
                nrOfUp = nrOfTrips2b(levels, up, down, destination, position + up, trips + 1, bestSofar);
    if(bestSofar > nrOfUp) bestSofar = nrOfUp;
    if (position - down >= 1)
                nrOfDown = nrOfTrips2b(levels, up, down, destination, position - down, trips + 1, bestSofar);
    return Math.min(nrOfUp, nrOfDown);
}
```

```
static int nrOfTrips2b(int levels, int up, int down, int destination) {
    return nrOfTrips2b(levels, up, down, destination, 1, 0, 30);
}
```

Med vår snyggare lösning

Dock går det alldeles utmärkt att effektivisera vår första snygga lösning också.

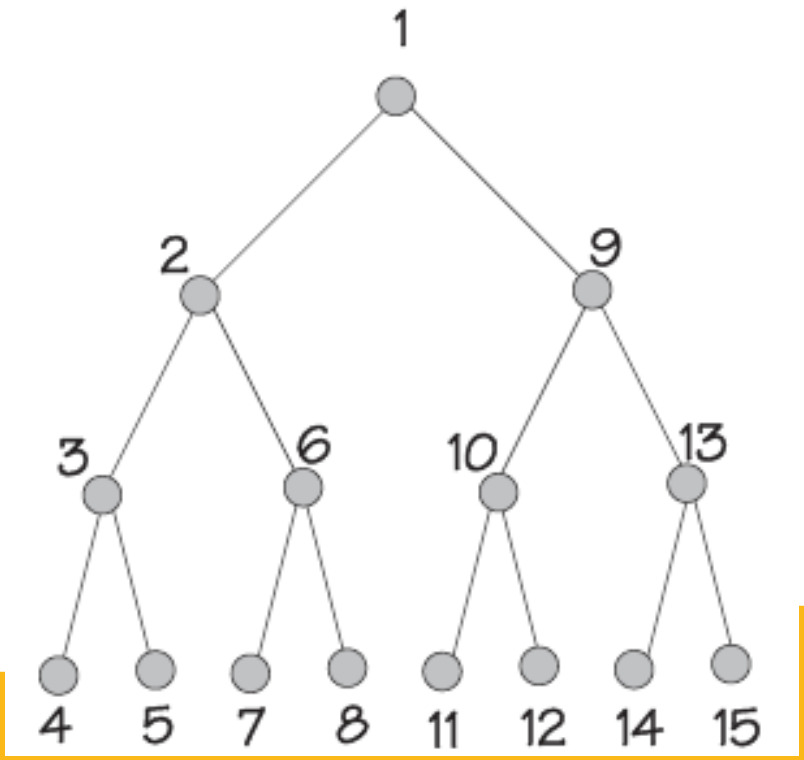
```
private static int nrOfTripsb(int levels, int up, int down, int destination, int position, int trips, int bestSofar) {
    calls++;
    if (position == destination) return 0;
    if (trips >= bestSofar-1) return Integer.MAX_VALUE / 2;
    int nrOfUp = Integer.MAX_VALUE/2, nrOfDown = Integer.MAX_VALUE/2;
    if (position + up <= levels)
                nrOfUp = 1 + nrOfTripsb(levels, up, down, destination, position + up, trips + 1, bestSofar);
    if(bestSofar > nrOfUp+trips) bestSofar = nrOfUp+trips;
    if(position - down >= 1)
                nrOfDown = 1 + nrOfTripsb(levels, up, down, destination, position - down, trips + 1,bestSofar);
    return Math.min(nrOfUp,nrOfDown);
}
```

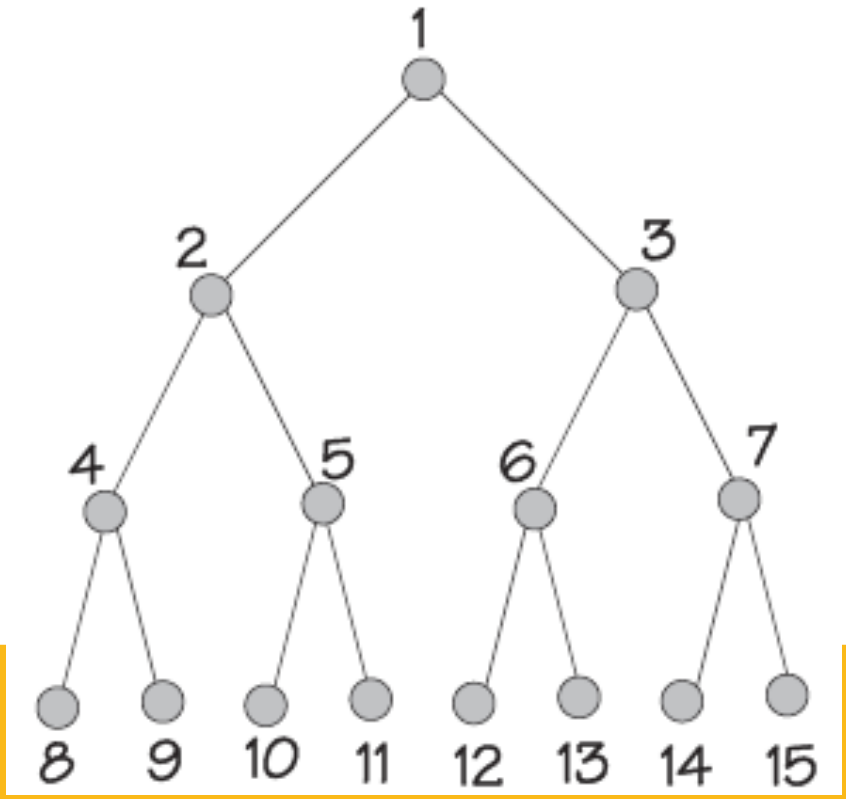
För vårat första exempel nrOfTrips(78, 15, 8, 35) gör första versionen 22126403 rekursiva anrop medans denna gör 45953 anrop.

Det kan vara frestande att åstadkomma detta med en medlemsvariabel (global variabel) men undvik helst detta!

Djupet kontra bredden först

Djupet först har för vissa problem nackdelen att vi måste sätta ett ofta godtyckligt maxdjup. Om vi vill hitta den lösning man når i färst antal steg blir den dessutom ineffektiv. Varför ska vi ner och gräva innan vi tittar om lösningen finns på en högre nivå?





Bredden först

Anledningen att vi får djupet först i vår rekursiva lösning är att våra funktionsanrop hamnar på en stack där det gäller att Last In First Out.

Om vi vill söka igenom lösningsträdet med bredden först behöver vi, just det en kö!

Stacken har vi så att säga gratis. Kön måste vi hantera själva.

Bredden först med hissen

n=1. Köa upp (4).

Kö: 4

Avköar n=4. Vi köar upp (7) och ner (3).

Kö: 7, 3

Avköar n=7. Vi köar ner (6)

Kö: 3, 6

Avköar n=3. Vi köar upp (6) och ner (2)

Kö: 6, 6, 2

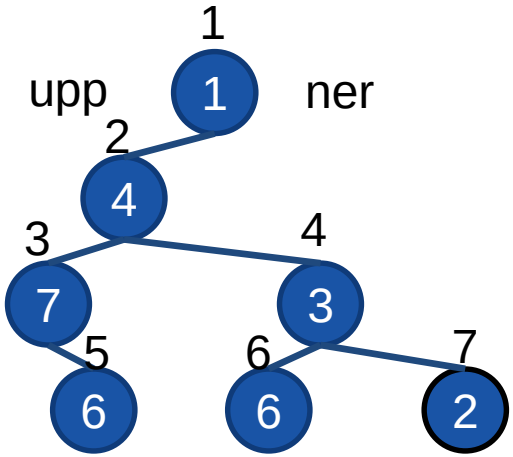
Avköar n=6. Vi köar ner (5)

Kö: 6, 2, 5

Avköar n=6. Vi köar ner (5)

Kö: 2, 5, 5

Avköar n=2.



Hur köar vi en plats i trädet?

De data som är relevanta för en plats i trädet är vilken våning befinner vi oss på och hur många resor vi har gjort.

För att kunna köa platser i trädet behöver vi alltså kunna hålla reda på dessa data.

Enklast skapar vi en klass:

```
private static class State {

    public int position, nrOfTrips;

    public State(int position, int nrOfTrips) {
        this.position = position;
        this.nrOfTrips = nrOfTrips;
    }
}
```

Hissen med bredden först

```
private static class State {

    public int position, nrOfTrips;

    public State(int position, int nrOfTrips) {
        this.position = position;
        this.nrOfTrips = nrOfTrips;
    }
}
```

```
private static int nrOfTrips3(int levels, int up, int down, int destination) {
    Queue<State> q = new ArrayDeque<>();
    State t = new State(1, 0);

    while (t.position != destination && t.nrOfTrips < 100) {
        if (t.position + up <= levels)
            q.offer(new State(t.position + up, t.nrOfTrips + 1));
        if (t.position - down >= 1)
            q.offer(new State(t.position - down, t.nrOfTrips + 1));
        t = q.poll();
    }
    return t.nrOfTrips;
}
```

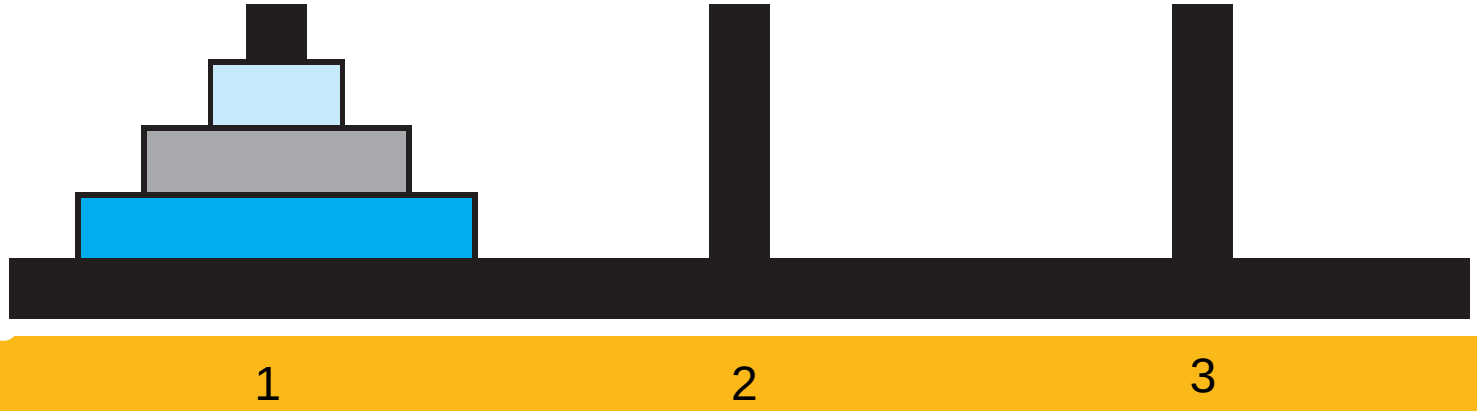
Observera hur lätt vi kunde skriva om problemet till iteration tack vare kön.
Antalet iterationer för nrOfTrips(78, 15, 8, 35) blir 1630!

Tornet i Hanoi

n brickor ska flyttas från pinne 1 till pinne 2.

Man får bara flytta en bricka i taget och bara översta brickan i en hög.

En bricka får aldrig ligga på en mindre bricka



Algoritm

Flytta n brickor från f till t (x tredje)

Om n>0

flytta n-1 brickor från f till x

flytta 1 bricka från f till t

flytta n-1 brickor från x till t

Kod

```
static void hanoi(int n, int f, int t, int x){
        if(n>0){
                hanoi(n-1,f,x,t);
                System.out.println(f+"->"+t);
                hanoi(n-1,x,t,f);
        }
}
```

anrop:hanoi(3,1,2,3) betyder alltså flytta 3 brickor från pinne 1 till pinne 2

Resultat:
1->2
1->3
2->3
1->2
3->1
3->2
1->2

Hur skulle du skriva en iterativ lösning?

Analys

$T(0) = 0$

$T(n) = 2T(n-1) + 1$

n	1	2	3	4	5	6
T(n)	1	3	7	15	31	63

$T(n) = 2^n - 1 = O(2^n)$

Enligt legenden skulle världen gå under när munkarna hade gjort tornet med 64 brickor:
$(2^{64} - 1)$ s $\approx$ 585 miljarder år

Backtracking

Backtracking är en klass algoritmer som

- bygger upp lösningskandidater till ett problem steg för steg
- överger en kandidat så fort den inte längre kan vara en lösning
- och då backar ett steg för att hitta en ny lösningskandidat.
- Detta gör den på ett sådant sätt att den aldrig prövar samma kandidat två gånger.

Backtracking är mycket effektivare än att pröva alla kombinationer där det är möjligt att kunna avgöra om ett förslag till dellösning är felaktig. I backtracking behöver man normalt endast en instans för att representera tillståndet hos lösningen och den kräver därför mycket lite minne.

Exempel på problem: sudoku, labyrint, korsord, schack och åtta damer.

Backtracking i en labyrint

- Gå så långt du kan tills du kommer ut eller det tar stopp.
- Om det tar stopp backa till närmsta förgrening och prova en annan väg du inte har provat.
- Om du provat alla backa ännu längre till nästa förgrening och prova en annan väg du inte har provat.
- osv

Algoritm

Vi ska här representera en labyrint som ett rutnät där varje ruta antingen är en vägg eller öppen. En av de öppna rutorna är dessutom start och en är mål.

```
hittaUt(ruta)
Markera ruta som korrekt väg
Om ruta är målet
        returnera true
annars
        Om ruta ovanför är öppen och obesökt
                Om hittaUt(ruta ovanför) returnera true
        Om ruta till höger är öppen och obesökt
                Om hittaUt(ruta till höger) returnera true
        Om ruta nedanför är öppen och obesökt
                Om hittaUt(ruta nedanför) returnera true
        Om ruta till vänster är öppen och obesökt
                Om hittaUt(ruta till vänster) returnera true
        Markera ruta som besökt
        returnera false
```

Observera att det endast krävs en instans av labyrinten i minnet.

Primitivt skal

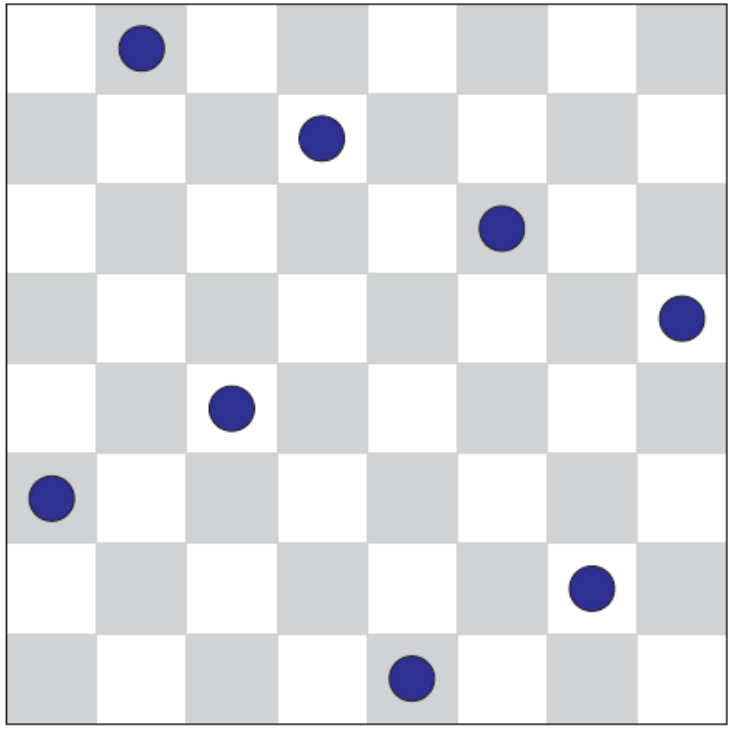
För att algoritmen ska fungera krävs att vi har vägg runt hela kanten utom vid målet. Ofta vill man ha starten precis vid kanten också och då kommer vår algoritm att riskera att försöka gå direkt ut ur labyrinten. För att slippa lägga till randkontroller i algoritmen har jag därför lagt till väggar runt hela labyrinten.

På kurssidan i canvas finns ett primitivt skal som läser in en labyrint från en textfil. Kvar är att skriva in själva algoritmen.

Observera att detta exempel på backtracking inte kan avfärda en dellösning förrän det tar stopp. Den är ändå effektivare än att prova alla lösningar eftersom vi aldrig provar att gå på samma väg två gånger. I nästa exempel kan vi avfärda förslag till lösningar i ett tidigt skede och lösningen blir mycket effektiv jämfört med att prova alla lösningar.

Åtta damer

Detta klassiska problem går ut på att placera 8 damer på ett schackbräde så att ingen dam står på samma rad, kolumn eller diagonal som en annan. På bilden ser du en sådan lösning. Vi ska skriva ett program som hittar alla lösningar med hjälp av backtracking.



Skissa på en algoritm

Den ska prova sig framåt tills det inte går. Då ska den backa till ett ställe där den har fler alternativ. Den kommer att vara rekursiv.

Tips. Ställ en dam på varje rad, en rad i taget. Ställ varje dam så att den står korrekt.

Algoritm

```
addQueen(rad)
för kolumn 1 till 8
        om kolumn möjlig
                boka platsen
                om rad==8
                        skriv ut lösning
                annars
                        addQueen(rad+1)
                avboka platsen
```

Anropas då med addQueen(1)

Obs att er faktiska metod behöver fler parametrar såsom brädet. Och ännu en gång observera att det endast krävs en instans av brädet.

Listiga tricks

Vi kan välja en 2d boolean matris som enda datastruktur och markera en drottning med true men det blir hyggligt jobbigt att kontrollera om en viss plats är möjlig.
Enklare då att förutom 2d-matrisen ha en array column, en array neDiagonal (↗) och en array nwDiagonal (↖) som håller reda på om dessa är upptagna.
Om column[3] är true betyder det då att den tredje kolumnen är upptagen.

Med diagonalerna är det lite trickigare.
Om neDiagonal[4] är true betyder att den diagonal där rad+kolumn=4 är upptagen
Alla element på en ne-diagonal har samma värde för rad+kolumn

Om nwDiagonal[4] är true betyder det att den diagonal där rad-kolumn+7=4 är upptagen.
Alla element på en nw diagonal har samma värde för rad-kolumn men lagras på plats rad-kolumn+7

(1, 1)	(1, 2)	(1, 3)	(1, 4)	(1, 5)	(1, 6)	(1, 7)	(1, 8)
(2, 1)	(2, 2)	(2, 3)	(2, 4)	(2, 5)	(2, 6)	(2, 7)	(2, 8)
(3, 1)	(3, 2)	(3, 3)	(3, 4)	(3, 5)	(3, 6)	(3, 7)	(3, 8)
(4, 1)	(4, 2)	(4, 3)	(4, 4)	(4, 5)	(4, 6)	(4, 7)	(4, 8)
(5, 1)	(5, 2)	(5, 3)	(5, 4)	(5, 5)	(5, 6)	(5, 7)	(5, 8)
(6, 1)	(6, 2)	(6, 3)	(6, 4)	(6, 5)	(6, 6)	(6, 7)	(6, 8)
(7, 1)	(7, 2)	(7, 3)	(7, 4)	(7, 5)	(7, 6)	(7, 7)	(7, 8)
(8, 1)	(8, 2)	(8, 3)	(8, 4)	(8, 5)	(8, 6)	(8, 7)	(8, 8)

Rekursion vs iteration

- En rekursiv lösning blir ofta kortare och ibland lättare både att skriva, läsa och felsöka. Dock kan det ibland vara svårt att debugga rekursiva lösningar just pga de rekursiva anropen.
- En rekursiv lösning kan spara minne i de fall man i en rekursiv lösning kan använda endast en instans för att spara tillståndet och ändra och ändra tillbaka före och efter det rekursiva anropet. Tänk t.ex. på åtta damer där det i en rekursiv lösning räcker med ett bräde för att utforska alla möjliga brädtillstånd.
- En rekursiv lösning blir ofta långsammare än en iterativ pga overhead - kostnaden för varje metodanrop. I de fall då vi inte kan spara plats kan den rekursiva också kräva mer minne eftersom vi måste spara alla lokala variabler och återhopps-adresser på stacken.
- En rekursiv lösning går normalt på djupet först och i vissa problem då söker av lösningsrymden på ett ineffektivt sätt och ibland kräver ett maxdjup.
- En rekursiv lösning som inte använder sig av dynamisk programmering (tabulering) beräknar ibland samma delproblem många gånger och kan då bli extremt in-effektiv.